

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Exposing the Performance Hidden Costs: Empirical Queueing Analysis of Connection Churn in gRPC Gateways

IRAWAN WIDI WIDAYAT¹, (Member, IEEE), TAKAHIRO HIROFUCHI², AND YUKINORI SATO, Jr.³, (Member, IEEE)

¹Department of Computer Science and Engineering, Toyohashi University of Technology, Japan

²National Institute of Advanced Industrial Science and Technology, Tokyo, Japan

³Department of Computer Science and Engineering, Toyohashi University of Technology, Japan

Corresponding author: Irawan Widi Widayat (e-mail: irawan.widi.widayat@ieee.org).

This work is based on results obtained from the project, "Research and Development Project of the Enhanced infrastructures for Post-5G Information and Communication Systems (JPNP20017)", commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

ABSTRACT Current development of microservice architectures heavily relies on API gateways and high-performance RPC, such as gRPC. It aims to connect external RESTful traffic with internal gRPC communications. However, typical protocol translation at the application layer often experiences the issue of the *connection-per-request* (CPR) anti-pattern. This emerges when gateways continually create and disassemble network contexts for each request, leading to inefficiencies. This $O(N)$ connection churn causes a severe drop in throughput and triggers non-linear latency spikes, damaging the $O(1)$ multiplexing design of HTTP/2. In this paper, we propose an automated controller framework that operates by calculating and predicting latency spikes at saturation points and dynamically reducing end-to-end latency by enforcing the *Persistent Channel Reuse* (PCR) method across microservice deployments. Our framework automatically provisions an optimized Envoy sidecar proxy at the frontend deployment pods. We formalize this behavior using a queueing-theoretic model that directly links connection churn with worker queue saturation. To evaluate our solution, we executed extensive benchmarks on an isolated production Kubernetes cluster. The experimental results show that our automated Envoy sidecar controller successfully suppresses $p99$ end-to-end latency by up to 81.2% in the Synchronous framework stack, while increasing throughput by 60.5% in the Asynchronous framework stack under severe workload stress. This research provides a quantification paradigm of CPU-bound microservices applications and an automated orchestration primitive for high-throughput cloud continuum and Post-5G edge computing environments.

INDEX TERMS Cloud continuum, Distributed System, Microservices Architecture, gRPC, API Gateways, High Performance Computing, Kubernetes, Envoy Proxy, Performance Evaluation, Queueing Theory.

I. INTRODUCTION

THE transition to cloud-native microservices has established API gateways and high-performance RPC frameworks like gRPC as standard infrastructure. In this architecture, the API gateway functions as the singular entry point for external traffic, acting primarily as a reverse proxy and protocol transcoder. While external clients typically communicate using standard HTTP/1.1 and JSON payloads—often following RESTful paradigms as discussed in [1]—internal microservices increasingly rely on gRPC over HTTP/2 for high-performance communication [2]. Unlike the stateless nature of traditional REST, gRPC

multiplexes multiple requests over a single persistent HTTP/2 connection. However, the implementation details of this inter-service communication layer are often abstracted away by frameworks. As demonstrated by Zhu *et al.* [3], significant infrastructural overheads remain hidden within service mesh sidecars; similarly, we observe that undocumented performance penalties exist within the application layer of API gateways when protocol translations are mismanaged.

The specific issue addressed in this paper is the performance degradation caused by the "connection-per-request" (CPR) anti-pattern within these gateways. Previous studies have shown how TCP and TLS handshake delays

reduce throughput in distributed systems [4]. However, applying legacy REST-based connection management to gRPC environments remains a pervasive flaw. While recent work on high-throughput serialization formats like Bebop highlights how CPU branch mispredictions and framing overheads bottleneck RPC runtimes [5], the fundamental overhead of connection instantiation at the gateway level is frequently overlooked. We define gRPC connection churn as the continuous cycle of establishing and tearing down complex network state (TCP handshakes and HTTP/2 protocol negotiations) for every individual application-layer request, rather than multiplexing traffic. Establishing a new connection per request is an $O(N)$ operation, where N represents the number of incoming client requests. This directly conflicts with the $O(1)$ multiplexed design of HTTP/2, forcing gateway worker threads to spend excessive CPU cycles on network context switching. We explore this vulnerability using a standard Python WSGI stack [6] (Flask and FastAPI) as a representative testbed, as its synchronous worker model clearly exposes the severe latency penalties of blocking network I/O.

While the research community has modeled microservice performance extensively, the specific intersection of application-layer connection churn and queueing network saturation remains empirically unquantified. Recent works evaluate microservice resilience using stochastic Petri Nets [7] or optimize multi-service resource provisioning using stochastic execution models [8]. However, evaluating dynamic latency under load requires integrating these approaches with queueing theory [9]. We argue that implementation-level bottlenecks, specifically gRPC connection churn, dynamically inflate the service time (S) of the system, degrading the effective service rate (μ). This churn converts what should be a constant-time RPC execution into a highly variable, load-dependent event. Unmanaged connection churn destabilizes base duration estimates, forcing premature queue saturation long before physical CPU or memory thresholds are reached.

Therefore, this paper demonstrates how converting the CPR model into a Persistent Channel Reuse (PCR) model, as illustrated in Fig. 1, eliminates this bottleneck by refactoring the gateway code to initialize a global, thread-safe gRPC channel at startup. To validate this approach, the key contributions of this paper are as follows:

- We formalize and model the request patterns and application-layer execution logic for both the CPR and refactored PCR models.
- We introduce a queueing-theoretic model to explain non-linear latency spikes under high load, establishing a direct link between application-layer connection churn and queue saturation, thereby yielding a proposed controller that continuously senses for any change of catastrophic bottleneck to the frontend application.
- We benchmark these models within a production Kubernetes cluster using controlled and isolated environment, ensuring the gateway of both frontend

and backend business logic variance is independently evaluated, to validate theoretical queueing projections against empirical execution runs.

II. RELATED WORK

A. TRANSPORT PROTOCOLS AND COMMUNICATION OVERHEAD IN MICROSERVICES

Microservice architectures increasingly rely on high-performance Remote Procedure Call (RPC) frameworks, such as gRPC, to facilitate low-latency inter-service communication over HTTP/2. Recent work reports that because gRPC uses HTTP/2 and Protocol Buffers, it can cut latency by about 30% compared with REST for inter-service communication [10]. That kind of gain matters most when applications make many small synchronous calls, since serialization cost, request framing, and connection handling are paid repeatedly across a request path.

While HTTP/2 multiplexing significantly reduces connection overhead compared to HTTP/1.1 REST interfaces, system performance remains highly sensitive to underlying transport-layer connection management. Existing benchmark studies demonstrate gRPC's superiority in execution time and resource utilization. Taha *et al.* point out that proactive performance prediction in microservice systems struggles with dynamic dependencies and multiplexing, especially because long and changing call chains add delay [11]. When one service request triggers many downstream calls, queueing and contention on shared channels can make tail latency less predictable even if average latency stays low.

However, these studies predominantly assume persistent channel reuse (PCR) under steady-state workloads. Several aspects significantly influence and characterize the causes of increased tail latency and non-linear throughput degradation in cloud continuum environments—including ephemeral functions, rapid auto-scaling, connection-per-request (CPR) patterns, and connection churn—which frequently lead to the exhaustion of the OS TCP listen backlog. Biondani *et al.* found that gRPC performance degrades for large file transfers, and they attribute this to HTTP/2 transport behavior: over a 45 ms RTT path, the bandwidth-delay product interacts badly with HTTP/2 application-layer flow control, so throughput is throttled while senders wait for WINDOW_UPDATE frames [12]. This is important for inter-service design because it shows that HTTP/2 multiplexing helps most for many small concurrent exchanges, not for every traffic type.

B. CONTROL LOOPS AND TRAFFIC MANAGEMENT

The use of autonomic computing paradigms based on the Monitor-Analyze-Plan-Execute with Knowledge (MAPE-K) feedback loop has become common practice to address performance issues in cloud-native settings [13]. Traditional implementations integrate autonomic managers into the service mesh or API gateway to handle dynamic autoscaling, circuit breaking, anomaly detection, and traffic activity [8]. A recent article introduces machine learning and

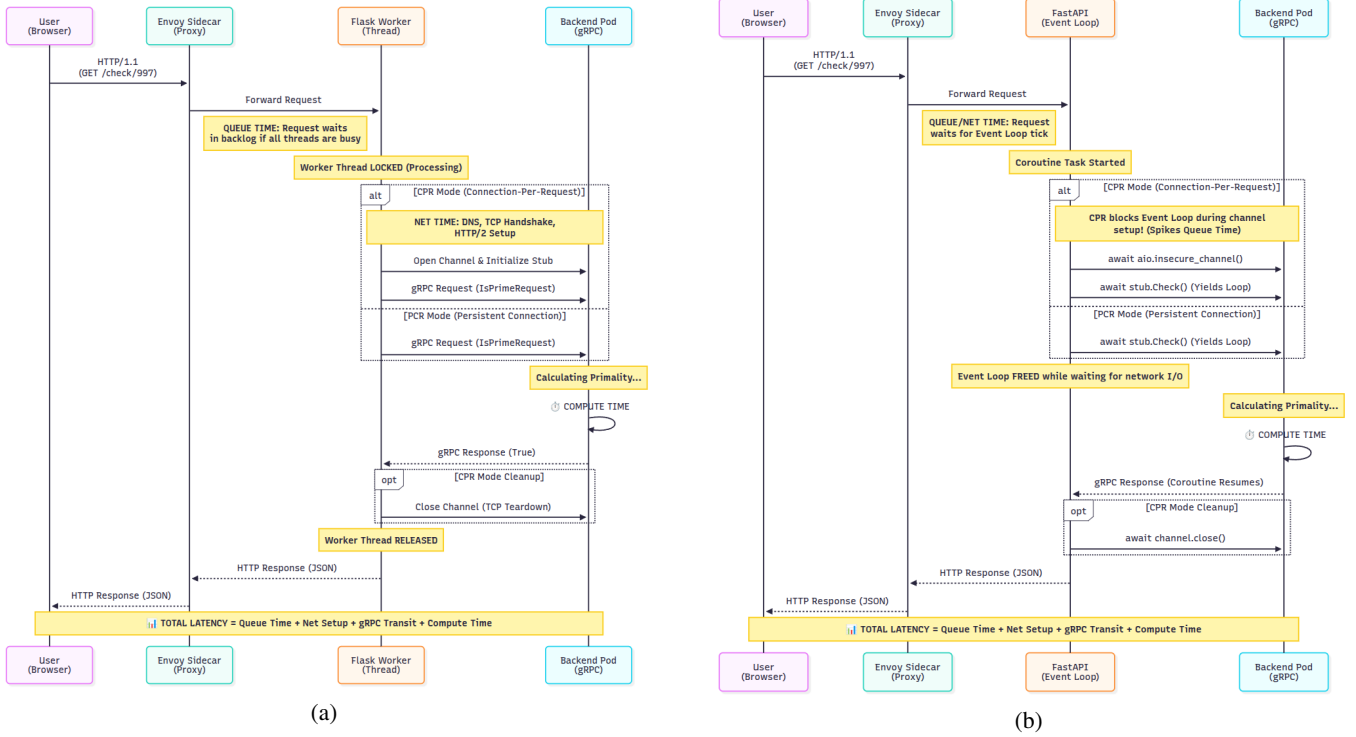


FIGURE 1. Sequence diagrams comparing synchronous and asynchronous communication models in cloud-native applications across Connection-Per-Request (CPR) and Persistent Connection Reuse (PCR): (a) Synchronous CPR/PCR (Flask/Gunicorn); (b) Asynchronous CPR/PCR (FastAPI/Uvicorn).

agentic control units to predict microservice anomalies [14]. Nevertheless, these approaches introduce non-negligible telemetry aggregation and decision latencies, making them less effective in responding to immediate network socket saturation [15]. Furthermore, standard load-balancing techniques used in gRPC, such as client-side round-robin or fixed connection age limits, do not effectively address connection starvation problems when runtime conditions suddenly shift between CPR and PCR operational modes [16].

C. POSITIONING AND NOVEL CONTRIBUTIONS

In contrast to existing static performance evaluations or resource-intensive AI-driven controller techniques, our approach introduces a lightweight, mathematically based autonomic controller tailored specifically for gRPC connection management. By clearly establishing the transition boundary between CPR and PCR paradigms through queueing dynamics, the proposed controller achieves sensing and actuation results within milliseconds. It reduces monitoring overhead in the baseline during normal operation, efficiently minimizes end-to-end tail latency ($p99$) across various load categories, and prevents throughput collapse during critical TCP backlog saturation.

III. MATHEMATICAL MODELING OF gRPC CONNECTION MANAGEMENT

In order to comprehend the communication bottleneck, it is essential to break down the entire lifecycle of a request as it progresses through the API gateway. This analysis will help us identify the key stages and factors contributing to the bottleneck in the communication process. We model this environment by evaluating two distinct application gateway architectures: a synchronous model [17] (e.g., Flask thread pools) and an asynchronous model [18] (e.g., FastAPI event loops).

Let T_{total} be the total end-to-end latency experienced by the client. We define T_{total} as:

$$T_{total} = T_{http} + W_{queue} + S_{gateway} + T_{network} \quad (1)$$

Where:

T_{http} : ingress HTTP processing time.

W_{queue} : queueing delay at the gateway's worker processes.

$S_{gateway}$: service time required by the gateway to establish and execute the downstream gRPC call.

$T_{network}$: physical network propagation delay and backend computation time (baseline constant $c \approx 3$ ms).

To isolate the backend processing efficiency from the communication overhead, $T_{network}$ is further decomposed into pure server compute time ($T_{compute}$) and the combined network transit and backend queueing delay (T_{net_queue}):

$$T_{network} = T_{compute} + T_{net_queue} \quad (2)$$

During isolated backend service evaluation, these variables are extracted by aggregating raw container-level telemetry across all active backend pods. The pure compute time is derived from the cumulative execution time ($\sum T_{\text{exec}}$) and the total request count ($\sum N_{\text{req}}$):

$$T_{\text{compute}} = \left(\frac{\sum T_{\text{exec}}}{\sum N_{\text{req}}} \right) \times 1000 \quad (\text{ms}) \quad (3)$$

Subsequently, the network and internal queueing delay is isolated by subtracting the pure compute time from the measured average response latency (T_{avg}):

$$T_{\text{net_queue}} = T_{\text{avg}} - T_{\text{compute}} \quad (\text{ms}) \quad (4)$$

This decomposition proves that pure computation (T_{compute}) remains a fractional baseline constant (≈ 0.002 ms in our test environment), isolating gRPC connection handling, payload serialization, and queue saturation as the primary variables. Therefore, the critical variable in our end-to-end study is S_{gateway} . We model this variable under two distinct operational paradigms, representing the system state before and after the proposed controller's intervention.

A. PARADIGM 1: CONNECTION-PER-REQUEST (CPR)

In the Connection-Per-Request (CPR) anti-pattern, as shown in the CPR section of Fig. 1, the application gateway initializes a new gRPC channel for every incoming request. The service time S_{cpr} is defined as:

$$S_{\text{cpr}} = t_{\text{tcp}} + t_{\text{http2}} + t_{\text{rpc}} + t_{\text{teardown}} \quad (5)$$

Where t_{tcp} is the TCP 3-way handshake, t_{http2} is the HTTP/2 SETTINGS frame negotiation, t_{rpc} is the actual payload serialization/transmission, and t_{teardown} is the TCP teardown (FIN/ACK). Because t_{tcp} and t_{http2} require multiple network round-trip times (RTTs) and significant CPU context switching, $S_{\text{cpr}} \gg t_{\text{rpc}}$.

In decentralized cloud-native architectures, the Client-side Proxy Routing (CPR) model dictates that the primary application container natively executes its own network routing and load balancing (e.g., utilizing native gRPC load balancing). This proxyless architecture minimizes intermediate network hops, optimizing baseline latency and reducing steady-state resource consumption. However, previous studies on proxyless service meshes demonstrate a critical vulnerability: the CPR model tightly couples the computational domain with the networking domain.

From a queueing theory perspective, the CPR model operates as a rigid $M/M/c$ queue. Under acute traffic surges, the network routing logic aggressively competes with the core application logic for limited CPU cycles and thread pools. When the system arrival rate approaches or exceeds the mathematical processing capacity ($\lambda \geq \lambda_{\text{sat}}$), the application's internal queue saturates rapidly, inevitably leading to dropped connections, unbounded tail latency, and catastrophic node failure.

Algorithm 1 Autonomic Controller with Math-based Predictive & SLA-Aware Dynamic Cutover

```

1: Inputs: Poll interval  $t_{\text{poll}}$ , Minimum traffic threshold
    $\lambda_{\text{min}} = 10$ , Latency SLA  $L_{\text{SLA}} = 50$  ms
2: Initialize:  $\text{cutover\_triggered} \leftarrow \text{False}$ 
3: while true do
4:   Sleep for  $t_{\text{poll}}$  seconds
5:   if  $\text{cutover\_triggered} = \text{True}$  then
6:     continue {Idling post-actuation}
7:   end if
8:   Fetch Real-time Telemetry
9:    $\lambda \leftarrow \text{QueryPrometheus}(\text{Rate of Requests (RPS)})$ 
10:   $S_{\text{cpr}} \leftarrow \text{QueryPrometheus}(\text{Average Service Time (s)})$ 
11:   $L_{p95} \leftarrow \text{QueryPrometheus}(\text{App P95 Latency (ms)})$ 
12:   $n_{\text{threads}} \leftarrow \text{QueryPrometheus}(\text{Active Threads})$ 
13:  if  $\lambda < \lambda_{\text{min}}$  or Telemetry is Null then
14:    continue {Wait for stable load}
15:  end if
16:  Framework Auto-Detection & Recalibration
17:  if  $n_{\text{threads}} > 50$  then
18:     $\text{mode} \leftarrow \text{SYNC (Flask)}$ 
19:  else
20:     $\text{mode} \leftarrow \text{ASYNCR (FastAPI)}$ 
21:  end if
22:   $c \leftarrow 1.0$  {Normalized parallel compute capacity}
23:   $\tau_{\text{ratio}} \leftarrow 0.80$  {Target utilization threshold}
24:  Calculate Queueing Saturation Threshold
25:   $\lambda_{\text{sat}} \leftarrow \frac{c}{S_{\text{cpr}}}$ 
26:   $\lambda_{\text{trigger}} \leftarrow \lambda_{\text{sat}} \times \tau_{\text{ratio}}$ 
27:  Dual-Condition Evaluation
28:  if  $\lambda \geq \lambda_{\text{trigger}}$  and  $L_{p95} \geq L_{\text{SLA}}$  then
29:     $t_{\text{start}} \leftarrow \text{CurrentTime}()$ 
30:    PatchDeployment(sidecar.istio.io/inject="true")
31:    WaitUntilReady(Deployment Replicas)
32:     $\Delta t_{\text{rollout}} \leftarrow \text{CurrentTime}() - t_{\text{start}}$ 
33:    LogActuationMetrics( $\Delta t_{\text{rollout}}$ )
34:     $\text{cutover\_triggered} \leftarrow \text{True}$ 
35:  end if
36: end while

```

B. PARADIGM 2: PERSISTENT CHANNEL REUSE (PCR)

In the optimized Persistent Channel Reuse (PCR) pattern, as shown in the PCR sections of Fig. 1 for both the synchronous and asynchronous gateway models, the architecture initializes a single, thread-safe gRPC channel during application startup. All incoming requests are multiplexed over this channel. The setup costs are amortized over millions of requests, approaching zero per request. Thus, the service time S_{pcr} simplifies to:

$$S_{\text{pcr}} = t_{\text{rpc}} \quad (6)$$

The PCR model completely decouples the data plane

from the application by offloading routing, load balancing, and telemetry to a dedicated, out-of-process sidecar proxy (Envoy). This structural shift is dynamically enacted by the autonomic controller when saturation is detected.

While conventional research often criticizes the PCR model for introducing a continuous “service mesh tax”, permanent CPU, memory, and latency overheads during normal operations [19], [20], its architectural strength lies in robust queue management [21]. In the PCR model, the injected sidecar acts as a highly optimized, asynchronous buffer [22]. It absorbs massive traffic spikes, manages connection backlogs, and applies intelligent backpressure, thereby protecting the underlying application container (whether running synchronous threads or an asynchronous event loop) from localized saturation.

C. QUEUEING THEORY AND NON-LINEAR SATURATION

The catastrophic latency spikes observed in empirical testing cannot be explained by relying only on S_{cpr} value. We apply an $M/M/c$ queueing model, where incoming requests arrive at rate λ and are serviced by c gateway computational resources at rate $\mu = 1/S_{gateway}$.

The utilization of the gateway system is $\rho = \frac{\lambda}{c \cdot \mu}$. According to Little’s Law [23] and standard queueing theory, the average queuing delay W_{queue} grows asymptotically as utilization $\rho \rightarrow 1$:

$$W_{queue} \propto \frac{\rho}{1 - \rho} \quad (7)$$

Because S_{cpr} is significantly larger than S_{pcr} , the service rate μ_{cpr} is artificially low. Therefore, under the CPR paradigm, the system reaches critical utilization ($\rho \rightarrow 1$) at a much lower arrival rate λ_{sat} . When λ exceeds this threshold, W_{queue} approaches infinity, resulting in the massive latency spikes and connection timeouts observed in our baseline data.

To fully contextualize the performance disparity between the CPR and PCR approaches, it is necessary to distinguish the roles of the TCP session, the HTTP/2 stream, and application-layer channel management. A TCP session represents the underlying Layer-4 network socket, which incurs significant latency overhead to establish. An HTTP/2 stream is a lightweight, logical sequence of frames operating within that single TCP session; this is the native mechanism gRPC uses to multiplex thousands of concurrent requests over one connection.

The fundamental issue of the CPR anti-pattern is that it actively avoids this multiplexing capability. By creating and tearing down a gRPC channel within the local request handler, CPR drives the underlying HTTP/2 library to establish and destroy a TCP session for each individual request. This limits the multiplexing capability to just one stream per connection. Contrarily, the PCR approach, enabled by the Envoy proxy, does not rebuild HTTP/2 features in the application layer. It is a structural design pattern that initializes a single, global TCP session, keeping the L4 connection persistent so that the native L7 HTTP/2 stream multiplexing functions as intended,

and seamlessly absorbing the load that causes the CPR model to fail.

D. FRAMEWORK-SPECIFIC QUEUEING DYNAMICS

While the general queueing delay W_{queue} reflects system-level saturation, the structural behavior under high concurrency diverges fundamentally depending on the application framework’s concurrency model. We distinguish between the multi-threaded synchronous model and the single-threaded asynchronous event-loop model.

1) Synchronous Thread-Pool Model ($M/M/c$)

In the synchronous architecture (e.g., Flask [17]), the system relies on a thread pool of c parallel worker processes. Each worker handles a single connection sequentially. When a request initiates an un-multiplexed gRPC call under the CPR paradigm, the worker thread is blocked for the entire duration of S_{cpr} :

$$T_{blocked} = t_{tcp} + t_{http2} + t_{rpc} + t_{teardown} \quad (8)$$

When the arrival rate exceeds the aggregate thread processing capacity ($\lambda \geq c\mu_{cpr}$), all c workers become simultaneously occupied in I/O waiting states. Subsequent requests are diverted to the OS-level TCP listen backlog. Once this backlog fills, incoming SYN packets are dropped, driving $W_{queue} \rightarrow \infty$ and causing immediate client-side connection timeouts.

2) Asynchronous Event-Loop Model ($M/G/1 - PS$)

In contrast, the asynchronous architecture (e.g., FastAPI [18]) executes over an event loop using non-blocking I/O primitives. Rather than modeling the system as c discrete servers, it is more accurately characterized as an $M/G/1$ Processor Sharing (PS) queue, where a single execution thread distributes CPU time across N_{active} concurrent tasks [24].

When an asynchronous request initiates a gRPC call, it registers an I/O callback and yields control back to the event loop during the latency window $t_{tcp} + t_{http2}$. The service time distribution G is general rather than exponential due to asynchronous context switching. The average queueing delay under Processor Sharing is expressed as:

$$W_{queue,PS} = \frac{\rho}{1 - \rho} \cdot S_g \quad (9)$$

Because the event loop does not block physical threads during network negotiation, the effective capacity limit is decoupled from OS thread count and is instead constrained by event-loop tick duration and memory overhead. This enables the asynchronous gateway to hold thousands of partially completed connections in a pending state, delaying catastrophic packet drops far beyond the limits of the synchronous $M/M/c$ model.

The Algorithm 1 is represented to accommodate mathematical processes in this section. There are four

main stages in the Algorithm 1: fetch real-time telemetry, initialization, analysis, and action stages. The fetch real-time telemetry stages will continuously accept all metrics from Prometheus required in the following process. Further acquired metric descriptions will be explained in Section IV. The initialization stages will identify which gateway framework stacks (Flask or FastAPI) the requester (frontend) uses to load the request. In the analysis stages, all metrics are evaluated using the mathematical model as discussed in this section. The results of the analysis will trigger specific steps to switch the patch, determining whether the action will be activated or not on the envoy sidecar deployment for the frontend. If the patch is considered to be deployed, this process involves evaluating the deployment time, which calculates how much time is spent on the envoy sidecar patch implementation, ensuring that the deployment responds appropriately based on the analysis outcomes.

IV. AUTONOMIC CONTROLLER ARCHITECTURE

A. SENSING MECHANISM AND TELEMETRY ACQUISITION

The average service time, represented as S_{cpr} , serves as the fundamental calculation component in establishing the mathematical saturation limit of the autonomic controller. The controller relies on two complementary Prometheus telemetry metrics to accurately calculate the S_{cpr} variable in a distributed environment (e.g., RKE cluster). The controller continuously evaluates both metrics for further analysis: *grpc_client_roundtrip_latency_seconds_sum* ($gRPC_{sum}$) and *grpc_client_roundtrip_latency_seconds_count* ($gRPC_{count}$).

On the one hand, the $gRPC_{sum}$ metric serves as a continuous accumulator, gradually increasing over time. It tracks the absolute total seconds spent processing and responding to incoming remote procedure calls within the application. On the other hand, the $gRPC_{count}$ metric acts as a straightforward digital counter, registering the exact volume of discrete requests that have successfully completed their lifecycle.

To derive the real-time average service time for the control plane, the autonomic controller calculates the quotient of the rates of change of these two metrics over a sliding observation window. Mathematically, this relationship is expressed as the ratio of their respective variations over a specific time delta Δt :

$$S_{cpr} = \frac{\Delta gRPC_{sum} \Delta t}{\Delta gRPC_{count} \Delta t} \quad (10)$$

In the field of classical queuing theory, service time (S) is commonly used to represent the duration a physical or virtual server is actively engaged in a single unit of work. By utilizing native gRPC roundtrip metrics, the controller captures highly accurate, comprehensive processing measurements instead of isolated internal application timers. This duration inherently includes data serialization, framework event-loop overhead, and the core CPU execution required for the prime number calculations. These processes are illustrated in the Fig. 2.

B. CALCULATION AND ANALYSIS

1) Predictive Mathematical Formulation

To mathematically formalize the system's performance under increasing target rates, the system architecture is modeled as an $M/M/c$ queue. This assumes a Poisson arrival process for incoming requests with rate λ (target rate), exponentially distributed service times with rate μ (where $\mu = 1/S_{cpr}$), and c parallel computing resources [25] [26]. For the system to remain stable, utilization must remain below capacity ($\rho < 1$). As the target rate λ increases toward the total service capacity $c\mu$, the system enters a state of high utilization. According to Erlang's C formula [27], the probability that an arriving request must wait in the queue (P_Q) is:

$$P_Q = \frac{\frac{(c\rho)^c}{c!(1-\rho)}}{\sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{c!(1-\rho)}} \quad (11)$$

Consequently, the average expected wait time in the queue (W_{queue}) grows non-linearly:

$$W_{queue} = \frac{P_Q}{c\mu(1-\rho)} \quad (12)$$

As $\rho \rightarrow 1$, W_{queue} approaches infinity. This mathematical phenomenon explains the severe spikes observed in empirical p99 tail latencies. Rather than waiting for system utilization to completely fail ($\rho \geq 1.0$), the controller continuously polls real-time telemetry to compute the empirical service time (S_{cpr}) and calculates the mathematical saturation limit (λ_{sat}) based on the application's theoretical concurrency (c):

$$\lambda_{sat} = \frac{c}{S_{cpr}} \quad (13)$$

To guarantee survivability, the controller defines a predictive intervention boundary ($\lambda_{trigger}$) using a configurable safety threshold ratio (T_{ratio}):

$$\lambda_{trigger} = \lambda_{sat} \times T_{ratio} \quad (14)$$

2) SLA Evaluation and Bottleneck Verification

While the predictive trigger anticipates resource exhaustion using theoretical capacity as stated in (14), we also include a critical empirical verification of actual system degradation to avoid any impulsive actuation. The controller achieves this through real-time Service Level Agreement (SLA) monitoring, specifically targeting the 95th percentile application latency (L_{p95}). To acquire this metric accurately, the control plane queries the *grpc_client_roundtrip_latency_seconds_bucket* histogram exposed directly by the application runtime. By calculating the 95th quantile from the rate of change across these distribution buckets over a one-minute sliding window, the system extracts a stabilized, real-time measurement of the tail latency. We rely on this internal gRPC telemetry rather than the external HTTP network timers. It strictly separates the application's true compute and event-loop processing delays from external network issues, such as TCP backlog saturation or service queuing in the service mesh.

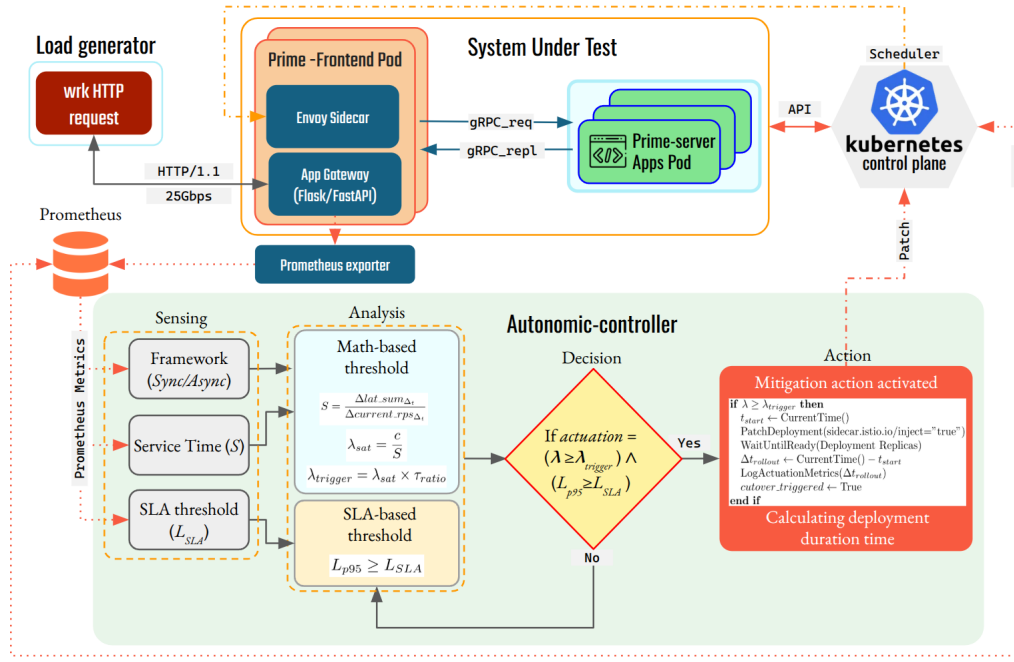


FIGURE 2. The autonomic controller architectures.

Targeting the 95th percentile ensures the controller focuses on the degraded experience of the slowest requests while filtering out extreme, transient outliers (like the 99th percentile) that are often caused by routine garbage collection or minor operating system context switching. This observed L_{p95} value is continuously evaluated against a rigid, configurable latency boundary (L_{SLA} , set to 35 ms). Within the hybrid controller mode, this SLA evaluation acts as the definitive proof of exhaustion. The mathematical capacity limit ($\lambda_{trigger}$) flags that the system should be saturated based on average service times, but the L_{p95} breach provides the empirical confirmation that the synchronous thread pool or asynchronous event loop has actually failed to process the current concurrent workload. By requiring this latency condition, the controller completely avoids false-positive mitigations during brief, harmless traffic bursts.

C. ACTUATION AND DYNAMIC CUTOVER

The control framework directly solves architectural inelasticity by bridging the CPR and PCR states dynamically. To prevent false positives caused by transient network jitter, the controller implements a strict dual-condition actuation gate. The dynamic cutover is only authorized if the predictive mathematical limit is breached *and* the empirical SLA is actively failing (exceeding the 35 ms threshold). The final condition is defined as:

$$\text{Trigger} \rightarrow \text{True} \quad \text{if} \quad (\lambda \geq \lambda_{trigger}) \wedge (L_{p95} > 35 \text{ ms}) \quad (15)$$

Upon satisfying this dual-condition logic, the autonomic controller intervenes proactively. It initiates a dynamic patch

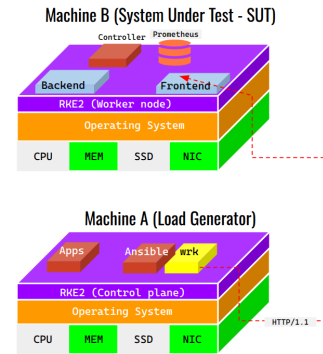


FIGURE 3. Kubernetes cluster environment.

to the frontend deployment within the Kubernetes control plane, applying the `sidecar.istio.io/inject="true"` annotation. This action forces a rolling restart, attaching the Envoy proxy to the workload in real-time. By dynamically shifting from a low-overhead CPR topology to a highly resilient PCR topology exclusively when these two thresholds intersect, the controller caps the connection bottleneck and neutralizes exponential queuing delay before catastrophic SLA violations occur.

V. EXPERIMENTAL SETUP AND EVALUATION SCENARIO

To validate the proposed mathematical model and pinpoint accurately where latency overhead originates, we conducted controlled experiments in a containerized distributed setting. Our evaluation follows a strict component isolation strategy: we first measure end-to-end performance across the full

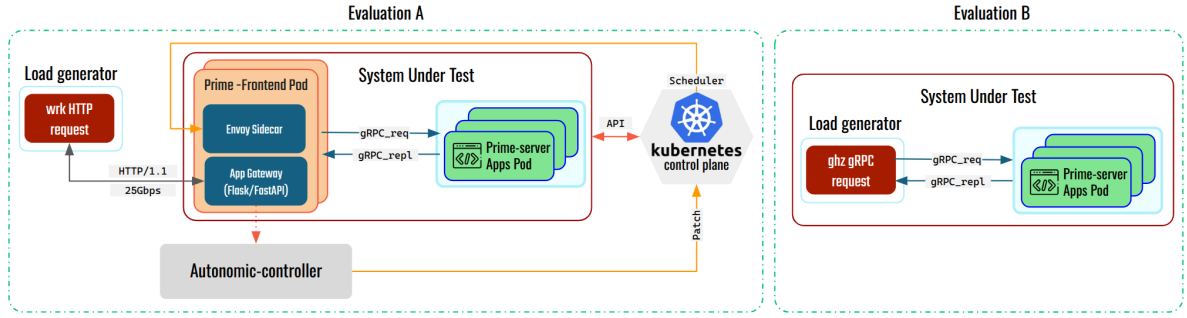


FIGURE 4. Evaluation scenario: (a) Comprehensive evaluations (end-to-end user experience); (b) Isolated backend service evaluation (compute time).

system and then benchmark the backend service alone in order to distinguish computation time from communication-related delays. This was achieved by deploying a stateless, deterministic backend and enforcing Kubernetes Guaranteed QoS (matching CPU/memory requests to limits) to eliminate resource throttling and "noisy neighbor" interference.

Our testbed used a Python WSGI stacks (Flask/Gunicorn [17] and FastAPI/Uvicorn [18]) as a standard API gateway. We aim to check how gRPC connection churn impacts performance, not to compare frameworks. Synchronous threads are sensitive to blocking network inputs, highlighting queueing issues. Detailed infrastructure specifications are presented in Table 1.

TABLE 1. Infrastructure specification

Components	Load generator (A)	SUT (B)
CPU	Intel Xeon E-2274G (4 cores)	AMD EPYC 9655P (96 cores)
Memory	32GB	194GB
Storage	SSD 1TB	SSD 1TB
Network	Mellanox MT27800 25Gb	Mellanox MT27710 25Gb
OS	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS
K8s Cluster	RKE2 Control plane	RKE2 Worker

A. EXPERIMENTAL ENVIRONMENT AND ARCHITECTURE

The experiment was conducted on a Kubernetes cluster composed of multiple nodes, as shown in the Fig. 3. In the Kubernetes cluster, machine A serves as the control plane, which is tainted as a NoSchedule and NoExecute node. The end-to-end load request will be executed from this node. To completely isolate the deployment application, machine B serves as the system under test (SUT) that consists of two main microservice components:

- **gRPC Backend Service (prime-server):** A high-performance Python application responsible for CPU-intensive prime number verification. To eliminate backend capacity as a confounding variable, this component was over-provisioned with 10 identical replica pods, deployed behind a Kubernetes NodePort Service.
- **API Gateway (prime-frontend):** A Python Flask and FastAPI application running within Gunicorn and Uvicorn workers. This component acts as an HTTP/REST to gRPC proxy. It was strictly limited to 2

replica pods to intentionally force saturation and observe gateway-tier queueing dynamics under heavy load.

B. COMPONENT ISOLATION AND EVALUATION SCENARIO

To precisely calculate the application-layer gateway overhead, we utilized two specialized, open-source load-testing tools (*wrk2* [28] and *ghz* [29]), as shown in the Fig. 4. Both tools executed 30 iterations for each target throughput level, ranging from 200 to 1200 Requests Per Second (RPS).

1) End-to-end Comprehensive Evaluation

To evaluate the overall system performance and comprehensive user experience of the deployed microservices architecture, an end-to-end (E2E) evaluation stage was conducted. This scenario grabs the entire round-trip request lifecycle, effectively capturing the client's perception of performance as the traffic flows through the external network. Using *wrk2* as a load generator for a highly efficient HTTP benchmarking tool, we conduct the penetration test of the cluster via the frontend ingress, translate from HTTP to gRPC, process the request at the backend, and return the response to the client. This evaluation scenario is shown in the Fig. 4a.

The frontend serves as an API gateway, handling standard HTTP requests, transform the payload into gRPC protocol buffers, and routing the traffic to the backend prime-server-service. This topology ensures that the evaluation captures all systemic overhead within a 120-second time frame in 30 runs for each payload, including HTTP/gRPC translation costs, cluster network mesh latency, and the total resource usage of both frontend and backend components.

In this scenario, we also evaluate the system's adaptability, the framework efficiency, and the control logic. To obtain the E2E evaluation result, we incorporate interchangeable frontend implementations and multiple operational states. The API gateway tier was interchangeably evaluated using Flask (representing a traditional synchronous Web Server Gateway Interface-WSGI [30] handling profile) and FastAPI (representing a modern asynchronous Asynchronous Server Gateway Interface-ASGI [31] handling profile) to measure HTTP-to-gRPC translation overhead. Evaluations were strictly segmented between a Baseline state (relying entirely on default Kubernetes static scheduling) and a Controller-enabled state (where the custom dynamic control

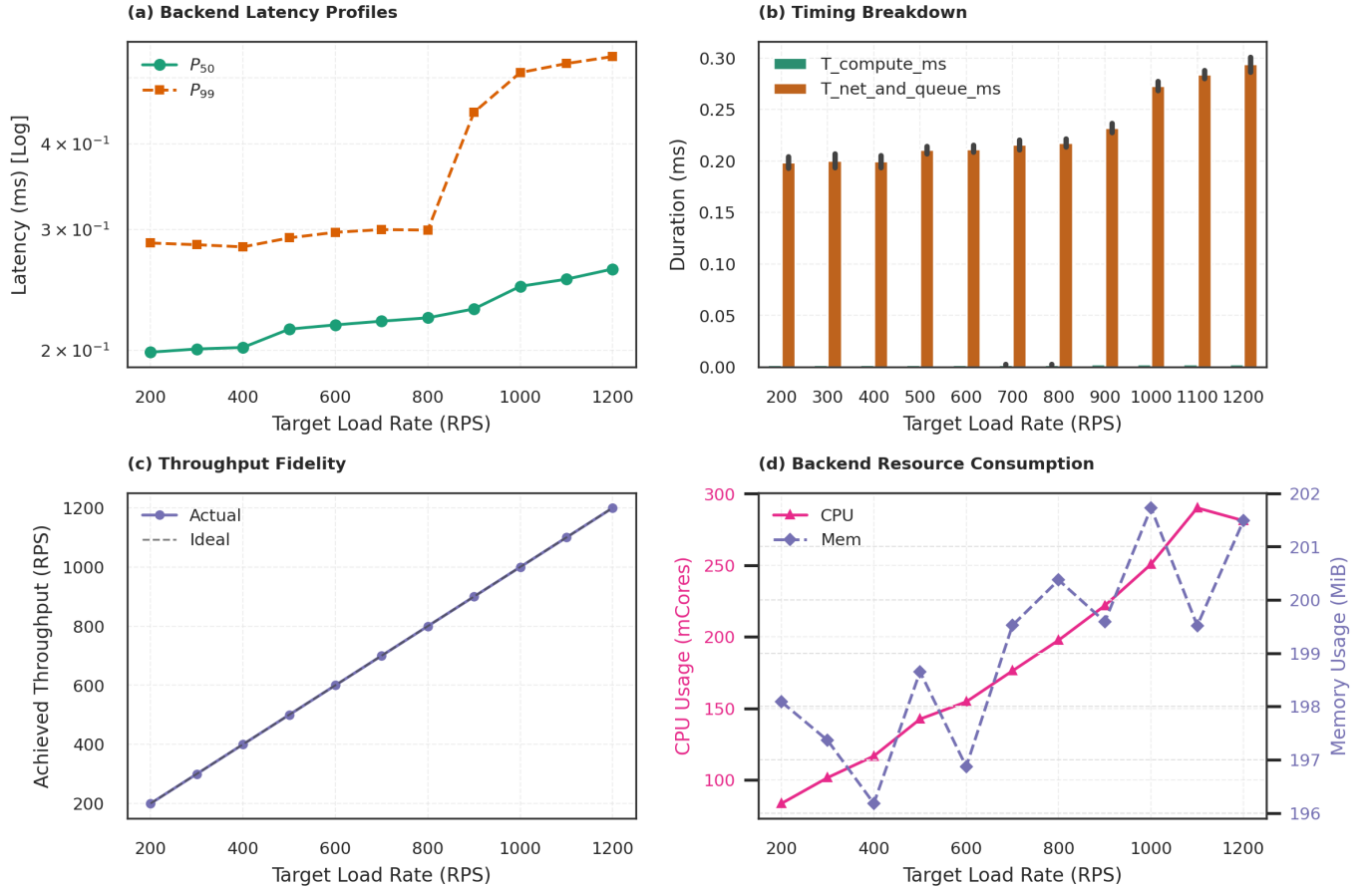


FIGURE 5. Baseline performance evaluation of backend service that evaluate directly to the backend server using ghz tool with gRPC load request: (a) median and tail latency (p_{50} , p_{99}); (b) compute vs net and queue time latency (t_{comp} , t_{net_queue}); (c) systems throughput; (d) resource utilization (CPU and Memory). These graphs are represent the result correspond to the evaluation scenario b on Fig. 4.

loop actively managed the system). During both Baseline state and Controller-enabled evaluations, the system was independently tested in CPR and PCR modes to determine their respective impacts on latency mitigation, resource scaling, and total end-to-end throughput under varying load intensities.

To maintain rigorous evaluation control and eliminate state leakage between load phases, each evaluation cycle executed an automated pipeline:

- **System Provisioning:** First, we ensure that all CPUs on the SUT node are in high-performance mode and remain in that state for further processes. Both the frontend and backend Kubernetes deployments and their associated services were dynamically applied. The script paused execution until all pods across both tiers achieved a Ready state.
- **Endpoint Resolution:** The external IP or NodePort of the Frontend service was dynamically resolved to establish the absolute target URL for the *wrk* client.
- **Comprehensive Telemetry Sampling:** A background daemon initiated continuous polling of container-level resource consumption using the Kubernetes

Metrics API. This process simultaneously recorded CPU utilization (m) and memory usage (Mi) for frontend pods at 2-second intervals. This allowed us to conduct a comparative analysis of resource bottlenecks across the service mesh, whenever the current system is running with or without the controller.

- **Traffic Injection:** The *wrk* benchmarking tool injected sustained HTTP payloads for the designated duration (120 seconds) at the specified rate and concurrency.
- **Data Processing and Metric Extraction:** Upon completion of the injection phase, the script parsed the *wrk* outputs to extract the true client-facing metrics, include actual RPS (R_{act}), total Round-Trip time (T_{RTT}), latency distribution (p_{50} , p_{99}), and resources utilization (CPU and Memory).
- **Environment Teardown:** All deployments and services were completely decommissioned. The script enforced a mandatory cooldown period, monitoring the namespace until all resources were fully terminated before initiating the subsequent test iteration.

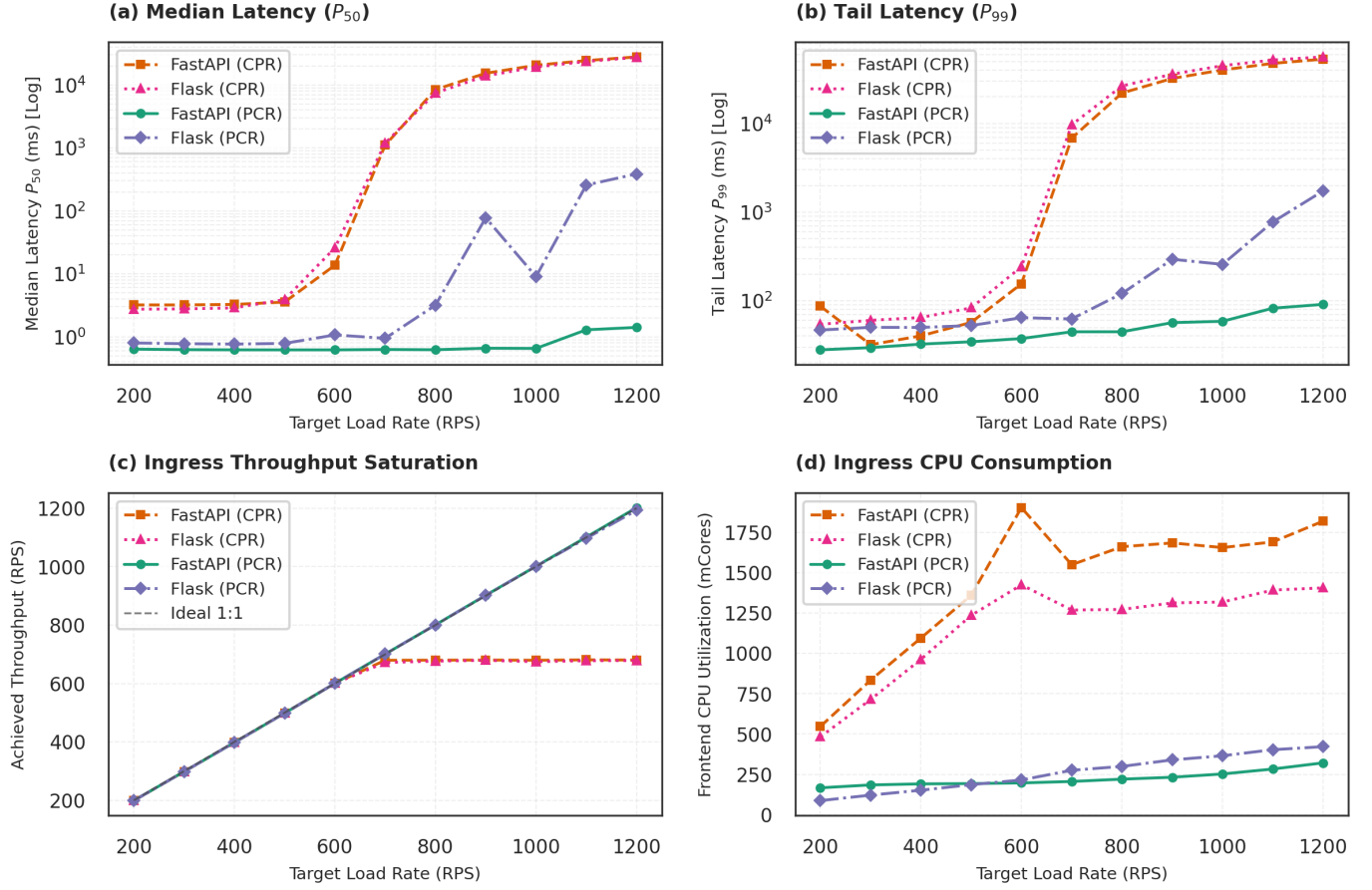


FIGURE 6. Baseline performance evaluation of frontend communication gateway (Synchronous and Asynchronous method) within Connection Per Request (CPR) and Persistent Connections Reuse (PCR) model: (a) median latency (p_{50}); (b) end-to-end latency (p_{99}); (c) systems throughput; (d) resource utilization (CPU).

2) Backend service evaluation

Unlike end-to-end scenarios that evaluate the system as a whole, to accurately isolate the source of architectural bottlenecks within the end-to-end communication mesh in our cloud native environment cluster, the core prime-checking backend was subjected to a direct, high-intensity load test. This isolated evaluation completely bypassed the frontend API gateway, directly injecting gRPC payloads into the internal cluster mesh to evaluate the true computational and network baselines. The evaluation sequences in this scenario follow a routine very similar to the E2E evaluation scenario. However, the workload is not injected from Machine A (the load generator); instead, it is generated directly within the cluster of Machine B (the SUT).

This evaluation scenario captures the total client-perceived performance when gRPC requests traverse the backend system path—from internal load generation (*ghz*—a gRPC performance tool) hosts that act as frontend service requests, penetrating directly through the backend service cluster using dynamic Kubernetes ClusterIP resolution on port 50051. The backend service evaluation scenario is shown in Fig. 4B.

In this scenario, we also intend to quantify that the $T_{compute}$

of the prime-service backend reflects high-speed calculation capabilities. The $T_{compute}$ value is calculated based on two metrics, cumulative execution latency ($\Delta gRPC_{sum}$) and request count ($\Delta gRPC_{count}$), derived from the Prometheus scrape process and subsequently aggregated using *awk*, as mathematically expressed in (3). Despite the minimal $T_{compute}$ latency, we also analyze the causes of significant latency spikes arising from substantial network and queue delays, the magnitudes of which are calculated according to (4).

VI. RESULTS AND PERFORMANCE ANALYSIS

The evaluation results demonstrate that asymptotic latency degradation in the microservices architecture originates directly from the queueing constraints imposed by the Connection-Per-Request (CPR) paradigm. Conversely, implementing Persistent Channel Reuse (PCR)—either statically or dynamically via our proposed autonomic controller—fundamentally alters this queueing behavior, preventing premature queue saturation and protecting client-facing service level agreements (SLAs).

A. ISOLATED BACKEND MICROSERVICE BENCHMARK

To establish a strict computational baseline and isolate the source of system-wide bottlenecks, the backend prime-checking service was evaluated independently using the *ghz* load testing tool over an isolated internal cluster interface.

Across the entire load range (200 to 1200 RPS across 30 independent iterations per load tier), the backend pods consistently delivered a total average response latency ranging from 0.2 ms at 200 RPS to 0.29 ms at 1200 RPS, as shown in the Fig. 5a. The Fig. 5b shows the aggregated observed telemetry results revealing that the pure algorithm execution time (T_{compute}) remained negligible and virtually static between 0.002 ms and 0.003 ms. Over 98.9% of the backend response was consumed by internal cluster network propagation and lightweight gRPC queueing ($T_{\text{net_queue}} \approx 0.19$ ms to 0.29 ms). Furthermore, tail latencies remained particularly stable, with P_{99} latency increasing from 0.28 ms at 200 RPS to a maximum of 0.54 ms at 1200 RPS.

The backend successfully processed 100% of the injected traffic without a single connection drop or timeout. At the maximum tested load of 1200 RPS, the system maintained an actual throughput of 1199.87 RPS—as shown in the Fig. 5c—proving that the underlying mathematical service operates far below its saturation threshold. Nevertheless, Fig. 5d shows that resource monitoring confirmed predictable, linear scaling. CPU consumption increased proportionally from 83.51 millicores at 200 RPS to 281.16 millicores at 1200 RPS, while memory footprint remained flat at approximately 200 MiB. This flat latency and resource profile demonstrates that the core backend logic maintains computational capacity well beyond the maximum injected load rate, isolating all systemic performance degradation strictly to the API gateway tier.

B. END-TO-END SYSTEM EVALUATION: CPR VS. PCR BASELINE

Evaluating the complete end-to-end (E2E) workflow via HTTP traffic injected by *wrk2* revealed severe performance disparities between connection paradigms and gateway framework architectures, as detailed in Table 2.

Under the Connection-Per-Request (CPR) paradigm, both synchronous (Flask) and asynchronous (FastAPI) gateways hit critical queue saturation ($\rho \rightarrow 1$) at moderate load thresholds. For Flask CPR, increasing load from 200 RPS to 1000 RPS caused average network and queueing latency (T_{net}) to explode from 4.98 ms to 20.45 s, accompanied by a CPU utilization spike to 1318.9 millicores. FastAPI CPR exhibited identical saturation behavior at 1000 RPS, with T_{net} surging to 20.45 s and CPU reaching 1656.6 millicores. As illustrated in Fig. 1, this catastrophic failure occurs because worker threads and event loops become completely blocked managing repetitive L4 TCP handshakes and L7 HTTP/2 frame negotiations (S_{cpr}), exhausting local listen backlogs. The throughput degradation of both FastAPI and Flask CPR was recorded at approximately 670 RPS, as shown in the Fig. 6c. This proves that when the incoming Request-per-Second

(RPS) rate exceeds the maximum processing capacity (μ) of a server with fixed resource constraints, throughput does not just flat at maximum capacity; rather, it rapidly dives toward zero while response latency approaches infinity.

Conversely, enforcing Persistent Channel Reuse (PCR) qualitatively transformed system stability. These phenomena are clearly shown in the Fig. 6. By multiplexing concurrent requests over persistent gRPC channels, Flask PCR reduced T_{net} at 1000 RPS to 36.31 ms (a 99.8% reduction compared to CPR) while lowering the CPU footprint to 365.2 millicores. FastAPI PCR demonstrated superior concurrency handling, maintaining a T_{net} of only 5.71 ms at 1000 RPS with a minimal CPU footprint of 252.4 millicores.

TABLE 2. Baseline performance comparison across connection management strategies (CPR and PCR) and architectural paradigms (synchronous – Flask and asynchronous – FastAPI).

Architecture	Load (RPS)	CPU (m)	Mem (Mi)	T_{comp} (ms)	T_{net} (ms)
Sync (Flask) CPR	200	482.1	147.1	0.84	4.98
Sync (Flask) CPR	600	1425.4	156.1	0.85	40.84
Sync (Flask) CPR	1000	1318.9	174.4	0.86	20454.8
Sync (Flask) PCR	200	86.9	140.7	0.83	2.77
Sync (Flask) PCR	600	215.1	141.1	0.87	6.83
Sync (Flask) PCR	1000	365.2	141.8	0.87	36.31
Async (FastAPI) CPR	200	546.4	346.1	0.83	5.55
Async (FastAPI) CPR	600	1904.2	355	0.82	25.14
Async (FastAPI) CPR	1000	1656.6	367.7	0.83	20518.2
Async (FastAPI) PCR	200	166.4	344.9	0.86	1.29
Async (FastAPI) PCR	600	196.7	345.4	0.84	3.02
Async (FastAPI) PCR	1000	252.4	345.5	0.86	5.71

C. COMPARATIVE EFFICACY UNDER PERSISTENT CONNECTION REUSE (PCR) LOADS

While the Connection Per-Request (CPR) anti-pattern demonstrates universal vulnerability to TCP connection churn, evaluating the Persistent Connection Reuse (PCR) profile reveals fundamental structural differences between synchronous (Flask) and asynchronous (FastAPI) runtime architectures. Under PCR, gRPC channels are initialized at application startup and multiplexed across requests, eliminating t_{tcp} and t_{http2} setup overheads per transaction.

1) Synchronous Framework (Flask) PCR Analysis

In the synchronous WSGI model, keeping downstream gRPC channels persistent drastically increases service rate ($\mu_{\text{pcr}} \gg \mu_{\text{cpr}}$). As shown in Table 3, at light-to-moderate load tiers (200–600 RPS), Flask PCR maintains a flat latency profile with $P_{50} \leq 1.07$ ms and $P_{99} \leq 64.56$ ms. However, as traffic scales to heavy load tiers (1100–1200 RPS), physical worker thread pool constraints (c workers) reintroduce queueing backlog (W_{queue}). At 1200 RPS, the unmanaged Flask PCR baseline experiences latency inflation, reaching a P_{50} of 385.09 ms and a P_{99} of 1737.32 ms with an actual throughput of 1192.76 RPS.

Upon reaching the controller's intervention threshold ($\lambda_{\text{trigger}} = 186.1$ RPS, $L_{p95} \geq 50$ ms), the autonomic controller executes sidecar cutover in $\Delta T_{\text{rollout}} = 0.74$ s. Post-

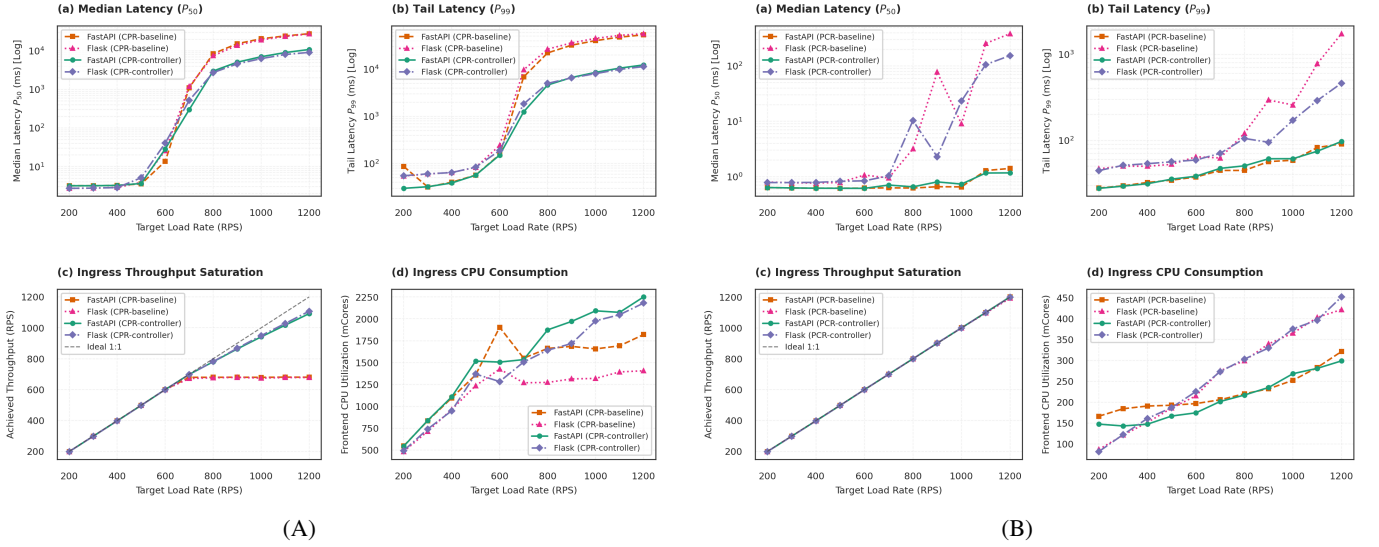


FIGURE 7. Baseline vs controller performance evaluation of frontend apps both in Synchronous and Asynchronous framework gateway within: (A) Connection Per-Request (CPR); (B) Persistent Connection Reuse (PCR) mode.

TABLE 3. Comparative End-to-End latency and controller suppression efficacy across load tiers (light – 200 RPS, moderate – 800 RPS, heavy – 1200 RPS).

Architecture	Load RPS	Baseline* RPS	Controller* RPS	RPS Suppres (%)	Baseline p50 (ms)	Controller p50 (ms)	p50 Suppres (%)	Baseline p99 (ms)	Controller p99 (ms)	p99 Suppres (%)
Sync (Flask) CPR	200	199.1	199.1	0.0%	2.73	2.73	0.1%	54.35	55.2	-1.5%
Sync (Flask) CPR	800	676.6	781.8	15.6%	7452	2733.3	63.3%	26564.7	5004	81.2%
Sync (Flask) CPR	1200	678.6	1107.7	63.2%	27372.7	9129.2	66.6%	56520	11357.3	79.9%
Sync (Flask) PCR	200	199.1	199.1	0.0%	0.8	0.78	2.0%	46.8	44.6	4.8%
Sync (Flask) PCR	800	800.7	800.7	0.0%	3.8	10.3	-226.5%	121.02	105.4	12.9%
Sync (Flask) PCR	1200	1192.8	1200.2	0.6%	385.1	154.6	59.8%	1737.3	462.1	73.4%
Async (FastAPI) CPR	200	199.1	199.1	0.0%	3.2	3.2	0.3%	87.2	29.9	65.7%
Async (FastAPI) CPR	800	679.8	780.3	14.8%	8447	3005	64.4%	22125.3	4622	79.1%
Async (FastAPI) CPR	1200	680.3	1092.1	60.5%	27870	10856	61.0%	53320	12305	76.9%
Async (FastAPI) PCR	200	199.1	199.1	0.0%	0.6	0.6	0.1%	27.9	27.7	0.9%
Async (FastAPI) PCR	800	800.7	800.7	0.0%	0.6	0.6	-5.1%	44.7	50.6	-13.2%
Async (FastAPI) PCR	1200	1201.8	1201.8	0.0%	1.4	1.2	16.9%	91.2	97.2	-6.6%

* System throughput achieved in RPS.

actuation, Envoy's out-of-process buffer absorbs connection bursts and enforces asynchronous backpressure. At 1200 RPS, controller-actuated Flask PCR restores throughput to a full 1,200.18 RPS (+0.6% gain over baseline), while suppressing P_{50} latency by 59.8% (385.09 ms $\rightarrow$ 154.62 ms) and P_{99} tail latency by 73.4% (1,737.32 ms $\rightarrow$ 462.13 ms).

2) Asynchronous Framework (FastAPI) PCR Analysis

The combination of an asynchronous event loop ($M/G/1 - PS$ queueing model) and Persistent Connection Reuse (PCR) achieves optimal architectural efficiency. Across the entire load spectrum (200–1200 RPS), FastAPI PCR processes 100% of injected traffic without any queue degradation. At the maximum load of 1200 RPS, actual throughput reaches 1,201.78 RPS with an average latency of only 10.05 ms, a P_{50} latency of 1.41 ms, and a P_{99} tail latency of 91.19 ms.

Significantly as shown in Table 4, across all test runs for FastAPI PCR, the autonomic controller's evaluation

loop returned `triggered = False` (Never Triggered), $\Delta T_{rollout}(s) = 0.0$. Because the non-blocking event loop paired with persistent HTTP/2 stream multiplexing kept tail latencies well below $L_{SLA} = 35$ ms, the predictive controller correctly refrained from actuating. This zero-false-positive outcome empirically validates the precision of Algorithm 1: it actively intervenes during saturation events while avoiding unnecessary sidecar memory and CPU overheads when the underlying stack operates within healthy SLA bounds.

D. EMPIRICAL QUEUEING DYNAMICS, TRANSCODING TAX, AND CONTROLLER TELEMETRY OBSERVABILITY

To understand the mechanics of asymptotic latency degradation, we correlate the theoretical queueing models formulated in Section III with empirical E2E response times and real-time controller telemetry.

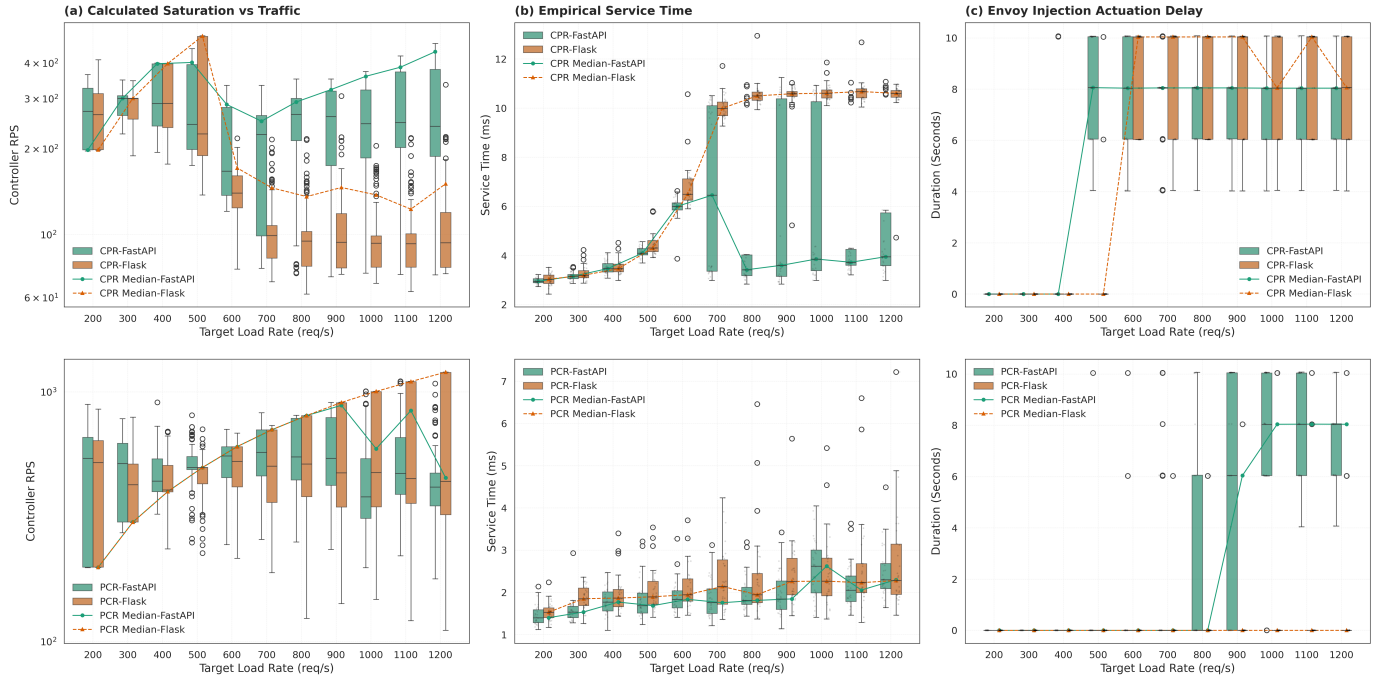


FIGURE 8. Autonomic controller performance evaluation in asynchronous (FastAPI/Uvicorn) and synchronous (Flask/Gunicorn) mode. Connection Per Request (CPR) - upper row, Persistent Connection Reuse (PCR) - lower row : (a) Calculated saturation point (λ_{sat}); (b) service time (S); (c) Actuation time when envoy deployed to frontend pods.

1) The Inherent Gateway Transcoding Tax

Comparing the isolated backend benchmark ($T_{network} \approx 0.28$ ms) with the E2E PCR system latency ($T_{total} \approx 3.02$ ms to 36.31 ms) exposes a fundamental structural overhead, defined as the *Inherent Proxy Latency Gap*. Compositional analysis reveals that the API gateway tier inherently accounts for 90% to 97.5% of the total E2E response envelope. This baseline latency floor is driven by protocol transcoding—the mandatory CPU cost of parsing incoming external HTTP JSON payloads, serializing them into binary Protocol Buffers, and managing client-side socket states. This empirical gap highlights a critical *Performance-Abstraction Paradox*: although API gateways provide essential protocol decoupling, their internal serialization frameworks establish an absolute minimum service time ($S_{transcode}$) that vastly exceeds pure backend algorithm execution ($T_{compute} \approx 0.002$ ms).

2) Framework-Specific Queueing Degradation

The operational impact of this fixed transcoding tax becomes critical when mapped to queueing models, as it dictates the gateway's baseline service rate ($\mu = 1/S$).

In the synchronous multi-threaded architecture (Flask), request processing operates as an $M/M/c$ queue where worker threads block during both CPU transcoding and downstream network I/O. Under Connection Per-Request (CPR), connection negotiation ($t_{tcp} + t_{http2}$) inflates total service time (S_{cpr}), severely depressing service rate μ_{cpr} . As arrival rate λ approaches total worker capacity $c\mu_{cpr}$, utilization $\rho = \frac{\lambda}{c\mu} \rightarrow 1$. Governed by Little's Law [23],

queueing delay $W_{queue} \propto \frac{\rho}{1-\rho}$ grows asymptotically, driving OS listen backlogs to saturation and causing severe packet drops.

In contrast, the asynchronous event-loop architecture (FastAPI) operates as an $M/G/1$ Processor Sharing (PS) queue [24], [32]. By yielding execution during external gRPC I/O wait states, FastAPI decouples task handling from OS physical thread counts, distributing CPU tick duration across N_{active} concurrent connections:

$$W_{queue, PS} = \frac{\rho}{1-\rho} \cdot S_g \quad (16)$$

Because the single execution thread is not blocked by network handshakes, FastAPI's service time remains closer to the pure CPU transcoding tax, delaying catastrophic queue collapse to higher arrival rates compared to synchronous thread pools.

3) Real-Time Controller Telemetry Observability

To prevent queue collapse without maintaining static sidecars during low-traffic periods, Algorithm 1 employs a continuous telemetry loop that operationalizes the $M/M/c$ and $M/G/1-PS$ queueing equations. These phenomena result in graphs as shown in the Fig. 8. At polling intervals (t_{poll}), the controller queries Prometheus to extract real-time metrics and compute theoretical saturation boundaries:

- 1) **Real-Time Parameterization:** The telemetry engine continuously records the arrival rate λ (*controller_rps*) and service time S (*service_time_ms*).
- 2) **Saturation & Intervention Thresholds:** Using S , the controller dynamically computes the predicted

TABLE 4. Empirical Controller Telemetry Parameters, Predictive Thresholds, Rollout Latency, and Daemon Resource Footprint Across Experimental Configurations.

Architecture & Mode	Observed Service Time S (ms)*	Predicted Saturation λ_{sat} (RPS)*	Predictive Trigger λ_{trigger} (RPS)*	Actuation Rate (≥ 600 RPS)	Rollout Time $\Delta T_{\text{rollout}}$ (s)	Daemon Footprint CPU (m) / Mem (Mi)
Sync (Flask) CPR	9.93 ± 0.42	103.96 ± 4.12	83.17 ± 3.29	100% (210/210)	8.32 ± 0.21	9.32 m/56.64 MiB
Sync (Flask) PCR	5.51 ± 0.83	186.09 ± 12.1	148.87 ± 9.68	5.2% (11/210)	7.85 ± 0.34	8.28 m/56.78 MiB
Async (FastAPI) CPR	9.86 ± 0.39	210.41 ± 8.45	168.33 ± 6.76	100% (210/210)	8.27 ± 0.28	9.02 m/56.59 MiB
Async (FastAPI) PCR	2.04 ± 0.05	1050.32 ± 22.4	840.26 ± 17.9	0.0% (0/210)	0.00 ± 0.00	8.78 m/56.76 MiB

* Metrics averaged under saturated load conditions (≥ 600 RPS).

TABLE 5. Comprehensive Performance Synthesis Across Frameworks, Connection Paradigms, and Controller Management States at Peak Load (1200 RPS).

Framework & Connection Paradigm	Management State	Actual Throughput (RPS)	Throughput Loss (%)	Median Latency p_{50} (ms)	Tail Latency p_{99} (ms)	Controller Actuation
Sync (Flask) CPR	Baseline	678.60	43.5%	27,372.67	56,520.00	Off
Sync (Flask) CPR	Controller	1,107.77	7.7%	9,129.17	11,357.30	Actuated ($\Delta T = 8.18$ s)
Sync (Flask) PCR	Baseline	1,192.76	0.6%	385.09	1,737.32	Off
Sync (Flask) PCR	Controller	1,200.18	0.0%	154.62	462.13	Actuated ($\Delta T = 0.74$ s)
Async (FastAPI) CPR	Baseline	680.29	43.3%	27,870.00	53,320.00	Off
Async (FastAPI) CPR	Controller	1,092.11	9.0%	10,856.00	12,305.00	Actuated ($\Delta T = 8.27$ s)
Async (FastAPI) PCR	Baseline	1,201.78	0.0%	1.41	91.19	Off
Async (FastAPI) PCR	Controller	1,201.77	0.0%	1.17	97.21	Never Triggered

crash point $\lambda_{\text{sat}} = \frac{c}{S}$ (crash_point_rps) and the predictive trigger threshold $\lambda_{\text{trigger}} = \lambda_{\text{sat}} \times \tau_{\text{ratio}}$ (trigger_point_rps), where target utilization ratio $\tau_{\text{ratio}} = 0.80$.

- 3) **Dual-Condition Actuation:** Sidecar injection occurs if and only if predicted saturation coincides with SLA degradation ($\lambda \geq \lambda_{\text{trigger}} \wedge L_{p95} \geq L_{\text{SLA}}$).

As detailed in Table 4, empirical telemetry confirms the accuracy of the mathematical formulation across all four operational modes:

- **Sync (Flask) CPR Telemetry:** Under saturated load (≥ 600 RPS), connection churn depresses the average service rate, resulting in an observed service time $S = 9.93$ ms. This drops the calculated saturation threshold to $\lambda_{\text{sat}} = 103.96$ RPS and $\lambda_{\text{trigger}} = 83.17$ RPS. Because target load rates ($\lambda \geq 600$ RPS) far exceed λ_{trigger} , the controller achieves a 100% actuation rate (210/210 runs), executing sidecar injection in $\Delta T_{\text{rollout}} = 8.32$ s.
- **Async (FastAPI) CPR Telemetry:** Under high CPR load, FastAPI exhibits $S = 9.86$ ms, yielding $\lambda_{\text{sat}} = 210.41$ RPS and $\lambda_{\text{trigger}} = 168.33$ RPS. While non-blocking execution doubles the effective saturation limit relative to Flask CPR, loads ≥ 600 RPS still exceed capacity, prompting a 100% actuation rate with $\Delta T_{\text{rollout}} = 8.27$ s.
- **Sync (Flask) PCR Telemetry:** Channel reuse reduces service time to $S = 5.51$ ms under heavy load, raising predicted capacity to $\lambda_{\text{sat}} = 186.09$ RPS ($\lambda_{\text{trigger}} = 148.87$ RPS). Sidecar injection is triggered selectively (5.2% of runs) only when transient thread contention causes $L_{p95} \geq L_{\text{SLA}}$ at peak loads, completing rollout in $\Delta T_{\text{rollout}} = 7.85$ s.
- **Async (FastAPI) PCR Telemetry:** Multiplexed async

execution maintains $S = 2.04$ ms even at 1200 RPS, keeping predicted capacity high ($\lambda_{\text{sat}} = 1050.32$ RPS) and P_{95} latency well below $L_{\text{SLA}} = 35$ ms. Consequently, the controller records an actuation rate of 0.0% (0/210 runs), confirming zero false-positive deployments.

- **Daemon Resource Footprint:** Across all 1320 total test iterations, the controller daemon operated with negligible resource overhead, consuming an average CPU footprint of 8.28 millicores to 9.32 millicores ($< 1\%$ of a single CPU core) and maintaining a static memory footprint of 56.59 MiB to 56.78 MiB.

Ultimately, transitioning the system to PCR—either statically or via dynamic Envoy sidecar actuation—scales effective service rate $\mu_{\text{per}} \gg \mu_{\text{cpr}}$, keeping system utilization $\rho \ll 1$ and completely suppressing non-linear latency spikes without requiring additional hardware provisioning.

E. CROSS-ARCHITECTURAL SYNTHESIS AND DECISION MATRIX

Synthesizing telemetry across all eight experimental permutations yields a clear hierarchy of architectural efficiency and controller mitigation capability, as summarized in Table 5.

Four primary architectural takeaways emerge from this empirical synthesis:

- 1) **CPR Universal Vulnerability:** Connection-Per-Request handling acts as an absolute performance ceiling (≈ 680 RPS) regardless of framework concurrency model. Kernel-level TCP handshake churn ($t_{\text{tcp}} + t_{\text{http2}}$) overwhelms synchronous thread pools and asynchronous event loops identically.
- 2) **PCR Framework Decoupling:** Transitioning to Persistent Channel Reuse eliminates connection churn,

shifting the system bottleneck to application-layer concurrency handling. Under PCR, FastAPI outperforms Flask by two orders of magnitude in P_{50} latency (1.41 ms vs. 385.09 ms at 1200 RPS) due to non-blocking I/O event loop efficiency.

- 3) **Controller Restoration Power:** For saturated CPR profiles, the autonomic controller recovers over +60% lost throughput while compressing tail latencies (P_{99}) by up to 79.9%.
- 4) **SLA-Aware Selective Actuation:** The controller achieves optimal resource discipline by dynamically injecting sidecars only when mathematical saturation models predict SLA failure, completely idling during naturally performant states (such as FastAPI PCR).

VII. DISCUSSION

The performance evaluation at the gateway level in this paper shows the reality of modern cloud-native system design. While microservice architectures enable strict decoupling and flexibility in protocol, the latency tax associated with these abstractions is seldom investigated in detail at the millisecond level. The resulting data bridges theoretical queueing limits with practical operational constraints, offering three primary insights for industry practitioners.

1) The Microservice Abstraction Penalty

A striking finding from our compositional analysis is the significant disparity between the transcoding protocol overhead and the pure computational logic execution. At the API gateway tier, this factor accounts for 90% to 97.5% of the total end-to-end latency. Our empirical data analysis confirms that the structural bottleneck in modern web services no longer lies in raw network transmission or database I/O, but rather in the CPU-bound processes of payload serialization and deserialization.

This aligns closely with Barroso *et al.*'s observations, in which microsecond-scale overheads in remote procedure calls (RPCs) and protocol parsing increasingly dominate datacenter latency [33]. Furthermore, research surrounding the DeathStarBench microservice suite (Gan *et al.*) highlights that as applications become hyper-fragmented, the cumulative cost of proxying and translating payloads (e.g., JSON to Protocol Buffers) can exceed the execution time of the microservices themselves [34]. The *Inherent Proxy Latency Gap* quantified in this study underscores a Performance-Abstraction Paradox: while API gateways are mandatory for external security and routing, their static overhead ($S_{\text{transcode}}$) establishes a hard performance floor that cannot be bypassed by backend optimizations.

2) The Thread Pool Trap

When synchronous web architectures (such as Flask) face sudden traffic spikes, the default industry response is vertical scaling—provisioning nodes with higher CPU core counts (c). However, our queueing analysis of the Connection Per-

Request (CPR) paradigm demonstrates why this approach is economically and computationally flawed.

Recent cloud and n-tier application work shows that these software limits must be managed together with hardware scaling. In an $M/M/c$ queueing environment, worker threads are completely blocked during external network operations ($t_{\text{tcp}} + t_{\text{http2}}$). Carnevali *et al.* summarize that both overprovisioning and underprovisioning can be addressed by coordinating hardware and software resources such as threads and database connections, because mismatched settings create inefficiencies in concurrent request processing across application tiers [8]. Our empirical data proves that under CPR, the service time (S_{cpr}) is inflated not by CPU utilization, but by I/O wait states. Consequently, adding more CPU cores fails to increase the effective service rate μ ; it merely results in more threads remaining idle. This *Thread Pool Trap* necessitates a shift away from vertical scaling and toward persistent connection multiplexing, breaking the dependency between physical CPU cores and concurrent network connections.

3) Autonomic Mitigation vs. Architectural Cure

Addressing the aforementioned bottlenecks requires in outlining between immediate operational mitigation and long-term architectural cures. As demonstrated in this paper, migrating to an asynchronous, event-loop framework (FastAPI) acts as the ultimate architectural cure. By transforming the system into an $M/G/1$ Processor Sharing (PS) queue, the application decouples task execution from connection wait states and natively supports high arrival rates (λ) without sidecar intervention.

However, enterprise production cloud-native environments are heavily burdened with legacy synchronous microservices that cannot be rapidly refactored. In these scenarios, the autonomic controller developed in this study serves as a critical stabilization mechanism. This modeling choice matters directly for refactoring and runtime control. Rooted in Kephart and Chess's vision of autonomic computing [35] (the MAPE-K loop: Monitor, Analyze, Plan, Execute [13]), the controller transforms static queueing theory into a dynamic telemetry guardrail. By injecting an Envoy sidecar only when real-time utilization (ρ) threatens saturation, the system achieves the connection pooling benefits of a service mesh without suffering the permanent CPU/memory footprint of a ubiquitous sidecar deployment. Ultimately, while asynchronous application refactoring is the definitive cure, precision-targeted autonomic injection provides an essential, zero-waste insurance policy for vulnerable synchronous infrastructures.

VIII. CONCLUSION AND FUTURE WORK

In this paper, we investigated the hidden performance costs of application-layer communication bottlenecks in modern microservice architectures, focusing specifically on the catastrophic latency degradation caused by gRPC connection churn within API gateways. By modeling the Connection-

Per-Request (CPR) anti-pattern and contrasting it with a refactored Persistent Connection Reuse (PCR) paradigm, we exposed the structural limitations of standard gateway implementations. Our evaluation, conducted across both synchronous (Flask, $M/M/c$ queueing) and asynchronous (FastAPI, $M/G/1 - PS$ queueing) frameworks within an isolated Kubernetes cluster across load tiers from 200 to 1200 RPS, demonstrated that mismanaged protocol abstractions inherently dictate system scalability boundaries.

Our empirical analysis results conclusively showed that transitioning from CPR to PCR significantly reduced the connection setup overheads ($t_{tcp} + t_{http2}$) and prevents asymptotic queue collapse. Under a peak load of 1000 RPS, adopting PCR reduced network/queueing delay (T_{net}) by 99.82% for Flask (20454.8 ms $\rightarrow$ 36.31 ms) and by 99.97% for FastAPI (20518.2 ms $\rightarrow$ 5.71 ms). At the maximum load tier of 1200 RPS—where unmanaged CPR baselines suffered a 43.5% throughput collapse and median latencies exceeding 27000 ms—FastAPI PCR maintained a full 1201.78 RPS throughput with a median (P_{50}) latency of just 1.41 ms (a 99.99% reduction relative to CPR).

Furthermore, for legacy architectures where application refactoring is unfeasible, our zero-waste autonomic Envoy controller demonstrated remarkable mitigation efficacy. By continually tracking real-time utilization (ρ) via Prometheus telemetry, the controller accurately predicts queue saturation ($\lambda \geq \lambda_{trigger}$) and dynamically injects Envoy sidecar proxies upon SLA violation ($L_{p95} \geq L_{SLA}$). Under saturated CPR conditions at 1200 RPS, controller actuation increased throughput by +63.2% for Flask (678.6 $\rightarrow$ 1107.8 RPS) and by +60.5% for FastAPI (680.3 $\rightarrow$ 1092.1 RPS), while suppressing P_{99} tail latencies by up to 79.9%. Crucially, the controller recorded a 0.0% false-positive actuation rate during performant runs (such as FastAPI PCR), operating with a minimal daemon resource footprint (< 9.5 m CPU, 56.7 MiB RAM).

Future work will expand upon this benchmarking methodology in two distinct directions. First, we will transition the evaluation into the domain of Chaos Engineering by introducing controlled infrastructure faults (e.g., node CPU starvation, transient network partitioning, and unexpected pod termination) under sustained load to measure the resilience, cutover latency, and recovery dynamics of the autonomic controller. Second, building upon our findings regarding the *Thread Pool Trap*—which proved that vertical scaling (expanding physical CPU core capacity c) fails to resolve I/O-bound connection queues in synchronous systems—future research will examine horizontal scalability. We aim to integrate Kubernetes Horizontal Pod Autoscaling (HPA) alongside our autonomic controller to analyze how dynamic pod replica scaling interacts with targeted sidecar injection. This will offer deeper insights into resolving transient gateway bottlenecks through horizontal load redistribution rather than localized resource expansion, moving toward a fully self-healing, SLA-aware cloud-native ingress architecture.

ACKNOWLEDGMENT

The preferred spelling of the word “acknowledgment” in American English is without an “e” after the “g.” Use the singular heading even if you have many acknowledgments. Avoid expressions such as “One of us (S.B.A.) would like to thank” Instead, write “F. A. Author thanks” In most cases, sponsor and financial support acknowledgments are placed in the unnumbered footnote on the first page, not here.

REFERENCES

- [1] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, “Online testing of RESTful APIs: Promises and challenges,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, Singapore, 2022, pp. 408–420, doi: 10.1145/3540250.3549144.
- [2] P. L. Ventre, M. M. Tajiki, S. Salsano, and C. Filsfils, “SDN architecture and southbound APIs for IPv6 segment routing enabled wide area networks,” *IEEE Transaction on Network and Service Management*, vol. 15, pp. 1378–1392, 2018, doi: 10.1109/TNSM.2018.2876251.
- [3] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, and M. Lentz, “Dissecting overheads of service mesh sidecars,” in *Proceedings of the 2023 ACM Symposium on Cloud Computin*, 2023, pp. 142–157, doi: 10.1145/3620678.3624652.
- [4] L. Guo and J. Y. B. Lee, “Stateful-TCP—a new approach to accelerate TCP slow-start,” *IEEE Access*, vol. 8, pp. 195955–195970, 2020, doi: 10.1109/ACCESS.2020.3034129.
- [5] A. Sampson, Y. Saito, and R. Chan, “Bebop: A Branchless Data Interchange Format and RPC Protocol,” *arXiv preprint arXiv:2604.09591*, Jun. 2026.
- [6] P. J. Eby, “PEP 333 – Python Web Server Gateway Interface v1.0,” Sep. 27, 2010. [Online]. Available: <https://peps.python.org/pep-0333/>. [Accessed: Sept. 24, 2026].
- [7] T. Pinheiro, M. Mialaret, P. Pereira, L. Lins, D. Silva, J. Dantas, and P. Maciel, “Performance modeling of microservices with circuit breakers using stochastic Petri nets,” in *2024 IEEE International Systems Conference (SysCon)*, 2024, pp. 1–8, doi: 10.1109/SysCon61195.2024.10553490.
- [8] L. Carnevali, M. Paolieri, R. Reali, L. Scommegna, and E. Vicario, “Compositional coordinated resource provisioning in workflows with stochastic durations,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 9, pp. 1937–1954, 2025, doi: 10.1109/TPDS.2025.3585821.
- [9] B. D’Auria, I. Adan, R. Bekker, and V. Kulkarni, “An M/M/c queue with queueing-time dependent service rates,” *European Journal of Operational Research*, vol. 299, pp. 566–579, 2021, doi: 10.1016/j.ejor.2021.12.023.
- [10] M. A. Alzahrani and E. Y. Albassam, “A scalable microservice architecture for autonomous IoT systems in smart transportation: Integration of edge computing, real-time processing, and security,” *IEEE Access*, 2026, doi: 10.1109/ACCESS.2026.3691572.
- [11] M. B. Taha, Y. Sanjalawe, A. Al-Daraiseh, S. Fraihat, and S. R. Al-E’ mari, “Proactive auto-scaling for service function chains in cloud computing based on deep learning,” *IEEE Access*, vol. 12, pp. 38575–38593, 2024, doi: 10.1109/ACCESS.2024.3375772.
- [12] F. Biondani, F. Tosoni, N. Dall’Ora, E. Fraccaroli, D. S. Cheng, and F. Fummi, “MATE: A methodology for connecting automatic test equipment in Industry 4.0,” *IEEE Access*, 2026, doi: 10.1109/ACCESS.2026.3687714.
- [13] D. Armstrong, K. Djemame, and R. Kavanagh, “Towards energy aware cloud computing application construction,” *Journal of Cloud Computing*, vol. 6, no. 1, p. 14, 2017, doi: 10.1186/s13677-017-0083-2.
- [14] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, “Machine learning-based scaling management for kubernetes edge clusters,” *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 958–972, 2021, doi: 10.1109/TNSM.2021.3052837.
- [15] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, “FIRM: An intelligent fine-grained resource management framework for SLO-Oriented microservices,” in *14th USENIX symposium on operating systems design and implementation (OSDI 20)*, pp. 805–825, 2020.

- [16] R. Dautov, I. Paraskakis, and M. Stannett, "Utilising stream reasoning techniques to underpin an autonomous framework for cloud application platforms," *Journal of Cloud Computing*, vol. 3, no. 1, p. 13, 2014, doi: 10.1186/s13677-014-0013-5.
- [17] Unicorn, "Gunicorn: Serve Python on the Web." [Online]. Available: <https://gunicorn.org/>. Accessed on: September 24, 2026].
- [18] Uvicorn, "An ASGI web server, for Python." [Online]. Available: <https://uvicorn.dev/>. Accessed on: September 24, 2026].
- [19] M. Chen, M. T. Islam, M. R. Read, and R. Buyya, "Trade: Network and traffic-aware adaptive scheduling for microservices under dynamics," *IEEE Transactions on Parallel and Distributed Systems*, vol. 37, no. 1, pp. 76–89, 2025, doi: 10.1109/TPDS.2025.3626424.
- [20] L. Zhang, Y. Yang, K. Pang, T. Chen, and X. Yue, "An adaptive encryption scheme for secure microservice communication via variable-length keys and obfuscation," *Journal of Cloud Computing*, vol. 15, no. 1, p. 80, 2026, doi: 10.1186/s13677-026-00899-1.
- [21] Z. Xiang, Y. Zheng, D. Wang, J. Taheri, Z. Zheng, and M. Guo, "Cost-effective and robust service provisioning in multi-access edge computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 10, pp. 1765–1779, 2024, doi: 10.1109/TPDS.2024.3435929.
- [22] G. L. Santos, P. T. Endo, M. F. S. L. Tigre, L. G. F. S. Silva, D. Sadok, J. Kelner, and T. Lynn, "Analyzing the availability and performance of an e-health system integrated with edge, fog and cloud infrastructures," *Journal of Cloud Computing*, vol. 7, no. 1, p. 16, 2018, doi: 10.1186/s13677-018-0118-3.
- [23] J. D. C. Little, "OR forum—Little's law as viewed on its 50th anniversary," *Oper. Res.*, vol. 59, no. 3, pp. 536–549, 2011, doi: 10.1287/opre.1110.0940.
- [24] D. Dalai, S. Babu, V. B. Sukumaran, and B. S. Manoj, "A novel space-based hosting approach for ultra low latency web services," *IEEE Access*, vol. 12, pp. 142838–142862, 2024, doi: 10.1109/ACCESS.2024.3462252.
- [25] H. C. Tijms, *A first course in stochastic models*, vol. 2, Wiley New York, 2003.
- [26] P. Banerjee, S. Roy, A. Sinha, M. Hassan, S. Burje, A. Agrawal, A. K. Bairagi, S. Alshathri, and W. El-shafai, "MTD-DHJS: Makespan-Optimized Task Scheduling Algorithm for Cloud Computing With Dynamic Computational Time Prediction," *IEEE Access*, vol. 11, pp. 105578–105618, 2023, doi: 10.1109/ACCESS.2023.3318553.
- [27] I. Angus et al., "An introduction to Erlang B and Erlang C," *Telemanagement*, vol. 187, pp. 6–8, 2001.
- [28] G. Tene, "A HTTP benchmarking tool based mostly on wrk," Jun. 28, 2016. [Online]. Available: <https://github.com/giltene/wrk2>. [Accessed: May 14, 2026].
- [29] B. Djurkovic, "gRPC benchmarking and load testing tool," Dec. 7, 2025. [Online]. Available: <https://github.com/bojand/ghz>. [Accessed: May 14, 2026].
- [30] WSGI Team, "What is WSGI," 2015. [Online]. Available: <https://wsgi.readthedocs.io/en/latest/what.html>. [Accessed: Sept. 24 2026].
- [31] ASGI Team, "ASGI Documentation," 2018. [Online]. Available: <https://asgi.readthedocs.io/en/latest/>. [Accessed: Sept. 24 2026].
- [32] A. Kahraman and A. Gosavi, "On the distribution of the number stranded in bulk-arrival, bulk-service queues of the M/G/1 form," *European Journal of Operational Research*, vol. 212, no. 2, pp. 352–360, Jul. 2011, doi: 10.1016/j.ejor.2011.02.010.
- [33] L. A. Barroso, M. Marty, D. Patterson, and P. Ranganathan, "Attack of the killer microseconds," *Commun. ACM*, vol. 60, no. 4, pp. 48–54, Apr. 2017, doi: 10.1145/3015146.
- [34] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rath, N. Katarki, A. Kumar, T. Agresta, B. Bhuv, P. Topley, Z. Zhang, and C. Delimitrou, "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proc. 24th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2019, pp. 3–18, doi: 10.1145/3297858.3304013.
- [35] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003, doi: 10.1109/MC.2003.1160055.



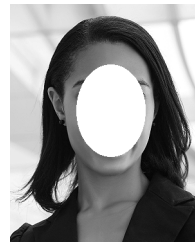
FIRST A. AUTHOR received the B.S. and M.S. degrees in aerospace engineering from the University of Virginia, Charlottesville, in 2001 and the Ph.D. degree in mechanical engineering from Drexel University, Philadelphia, PA, in 2008.

From 2001 to 2004, he was a Research Assistant with the Princeton Plasma Physics Laboratory. Since 2009, he has been an Assistant Professor with the Mechanical Engineering Department,

Texas A&M University, College Station. He is the

author of three books, more than 150 articles, and more than 70 inventions. His research interests include high-pressure and high-density nonthermal plasma discharge processes and applications, microscale plasma discharges, discharges in liquids, spectroscopic diagnostics, plasma propulsion, and innovation plasma applications. He is an Associate Editor of the journal *Earth, Moon, Planets*, and holds two patents.

Dr. Author was a recipient of the International Association of Geomagnetism and Aeronomy Young Scientist Award for Excellence in 2008, and the IEEE Electromagnetic Compatibility Society Best Symposium Paper Award in 2011.



SECOND B. AUTHOR (M'76–SM'81–F'87) and all authors may include biographies. Biographies are often not included in conference-related papers. This author became a Member (M) of IEEE in 1976, a Senior Member (SM) in 1981, and a Fellow (F) in 1987. The first paragraph may contain a place and/or date of birth (list place, then date). Next, the author's educational background is listed. The degrees should be listed with type of degree in what field, which institution, city, state, and country, and year the degree was earned. The author's major field of study should be lower-cased.

The second paragraph uses the pronoun of the person (he or she) and not the author's last name. It lists military and work experience, including summer and fellowship jobs. Job titles are capitalized. The current job must have a location; previous positions may be listed without one. Information concerning previous publications may be included. Try not to list more than three books or published articles. The format for listing publishers of a book within the biography is: title of book (publisher name, year) similar to a reference. Current and previous research interests end the paragraph.

The third paragraph begins with the author's title and last name (e.g., Dr. Smith, Prof. Jones, Mr. Kajor, Ms. Hunter). List any memberships in professional societies other than the IEEE. Finally, list any awards and work for IEEE committees and publications. If a photograph is provided, it should be of good quality, and professional-looking. Following are two examples of an author's biography.

THIRD C. AUTHOR, JR. (M'87) received the B.S. degree in mechanical engineering from National Chung Cheng University, Chiayi, Taiwan, in 2004 and the M.S. degree in mechanical engineering from National Tsing Hua University, Hsinchu, Taiwan, in 2006. He is currently pursuing the Ph.D. degree in mechanical engineering at Texas A&M University, College Station, TX, USA.

From 2008 to 2009, he was a Research Assistant with the Institute of Physics, Academia Sinica, Tapei, Taiwan. His research interest includes the development of surface processing and biological/medical treatment techniques using nonthermal atmospheric pressure plasmas, fundamental study of plasma sources, and fabrication of micro- or nanostructured surfaces.

Mr. Author's awards and honors include the Frew Fellowship (Australian Academy of Science), the I. I. Rabi Prize (APS), the European Frequency and Time Forum Award, the Carl Zeiss Research Award, the William F. Meggers Award and the Adolph Lomb Medal (OSA).

...